OPERATING-SYSTEM SERVICES



OPERATING-SYSTEM SERVICES

- **Execution of programs and services to programs and users**
- **Provide functions :**
  - **User interface**
  - **Program execution**
  - **I/O operations**
  - **File-system manipulation**
  - **Communications**
  - **Error detection**

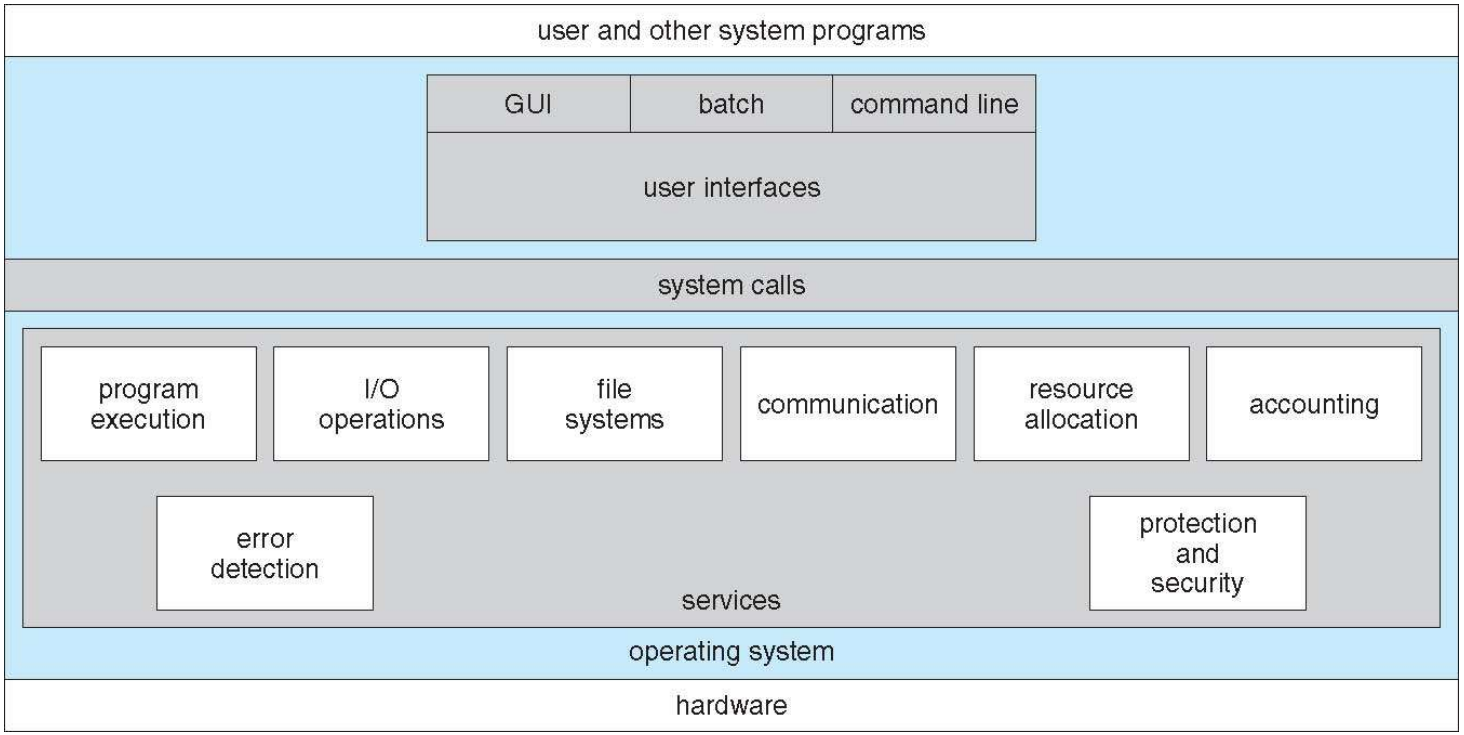


OPERATING-SYSTEM SERVICES (CONT...)

- OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation**
  - **Logging**
  - **Protection and security**



OPERATING-SYSTEM SERVICES



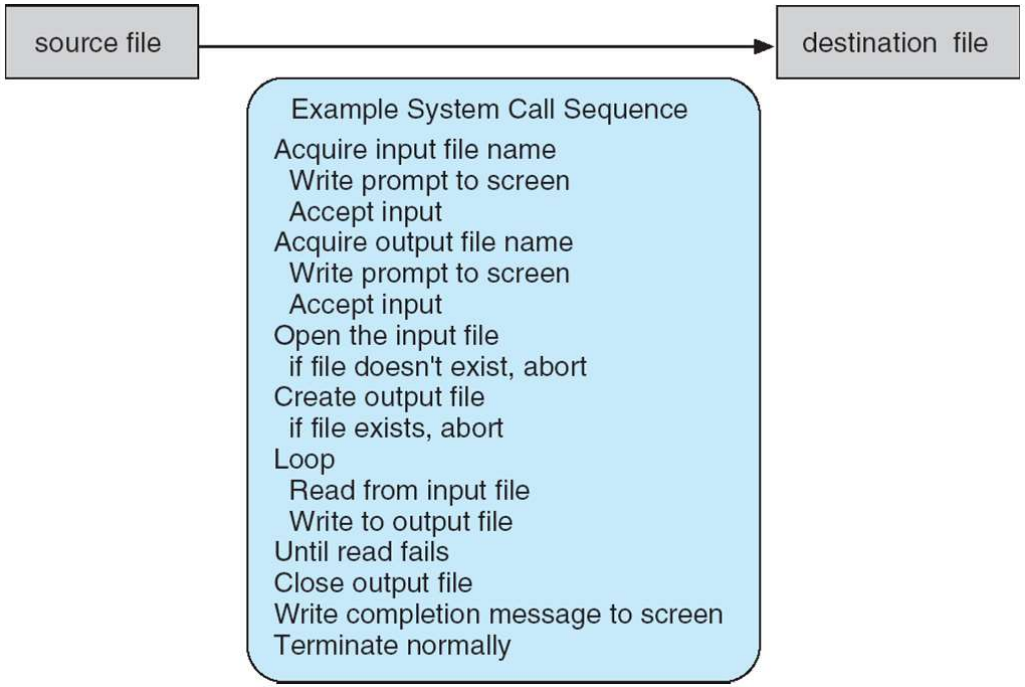
SYSTEM CALLS

- **Programming interface to the services provided by the OS**
- **Typically written in a high-level language (C or C++)**
- **Mostly accessed by programs via a high-level Application Programming Interface (API)**



EXAMPLE OF SYSTEM CALLS

- System call sequence to copy the contents of one file to another file



EXAMPLE OF STANDARD API

- consider the read() function that is avail-able in UNIX and Linux systems. The API for this function is obtained from
- the man page by invoking the command

  man read
- A description of this API appears below:

```
#include <unistd.h>

ssize_t     read(int fd, void *buf, size_t count)
```

return value — function name — parameters

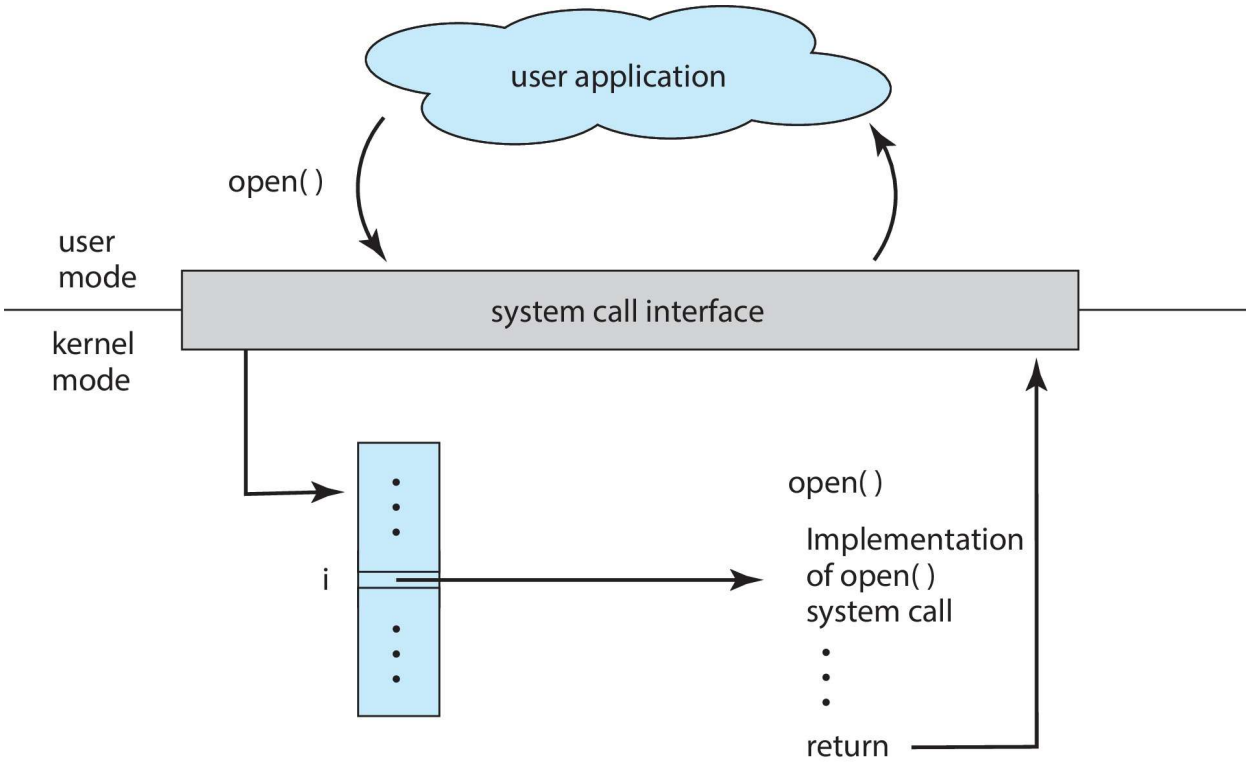


SYSTEM CALL IMPLEMENTATION

- **Typically, a number is associated with each system call**
  - **System-call interface maintains a table indexed according to these numbers**
- **The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values**
- **The caller need know nothing about how the system call is implemented**
  - **Just needs to obey API and understand what OS will do as a result call**
  - **Most details of OS interface hidden from programmer by API**
    - **Managed by run-time support library (set of functions built into libraries included with compiler)**



API – SYSTEM CALL – OS RELATIONSHIP

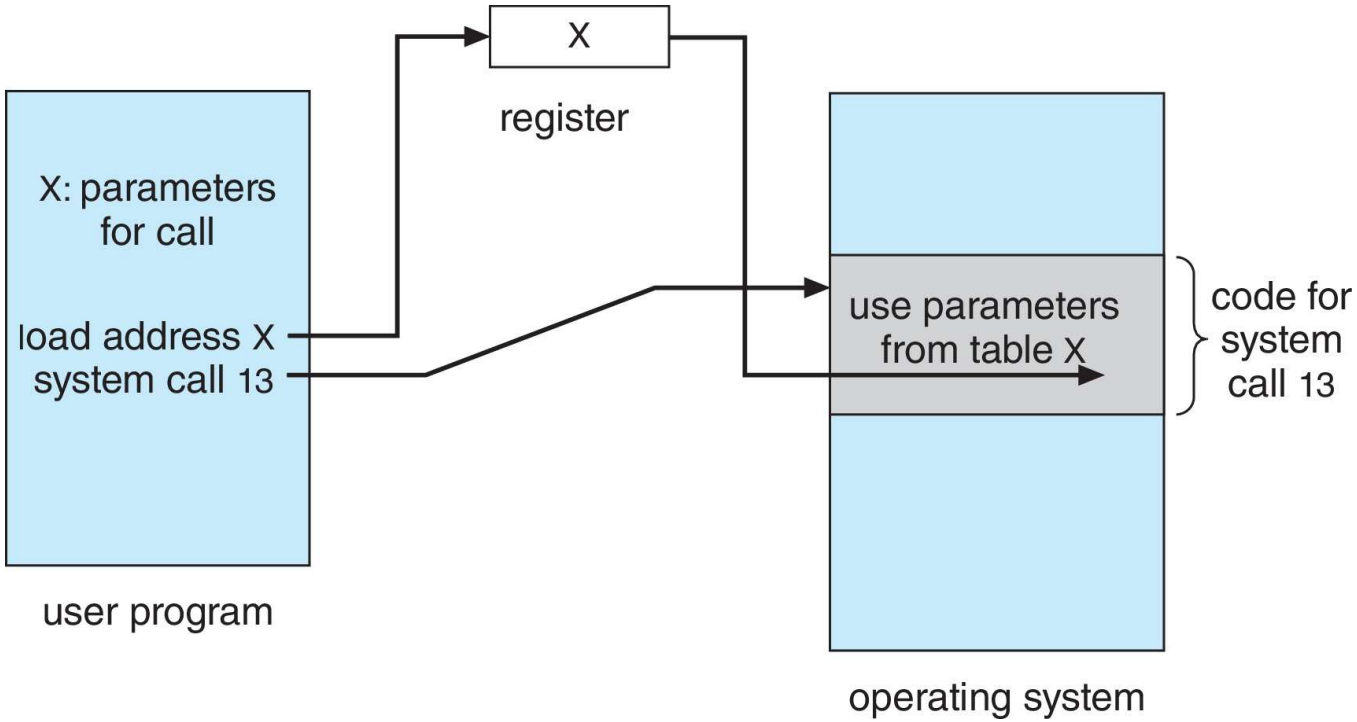


SYSTEM CALL PARAMETER PASSING

- **Methods used to pass parameters to the OS**
  - **Simplest: pass the parameters in registers**
    - **Parameters stored in a block, or table, in memory, and address of block**
    - **Parameters placed, or pushed, onto the stack**
    - **Block and stack methods**



PARAMETER PASSING VIA TABLE



TYPES OF SYSTEM CALLS

- **Process control**
  - **create process, terminate process**
  - **end, abort**
  - **load, execute**
  - **get process attributes, set process attributes**
  - **wait for time**
  - **wait event, signal event**
  - **allocate and free memory**
  - **Dump memory if error**
  - **Debugger for determining bugs, single step execution**
  - **Locks for managing access to shared data between processes**



TYPES OF SYSTEM CALLS (CONT.)

- **File management**
  - **create file, delete file**
  - **open, close file**
  - **read, write, reposition**
  - **get and set file attributes**
- **Device management**
  - **request device, release device**
  - **read, write, reposition**
  - **get device attributes, set device attributes**
  - **logically attach or detach devices**



TYPES OF SYSTEM CALLS (CONT.)

- **Information maintenance**
  - **get time or date, set time or date**
  - **get system data, set system data**
  - **get and set process, file, or device attributes**

- **Communications**
  - **create, delete communication connection**
  - **send, receive messages if message passing model to host name or process name**
    - **From client to server**
  - **Shared-memory model create and gain access to memory regions**
  - **transfer status information**
  - **attach and detach remote devices**



TYPES OF SYSTEM CALLS (CONT.)

- **Protection**
  - **Control access to resources**
  - **Get and set permissions**
  - **Allow and deny user access**



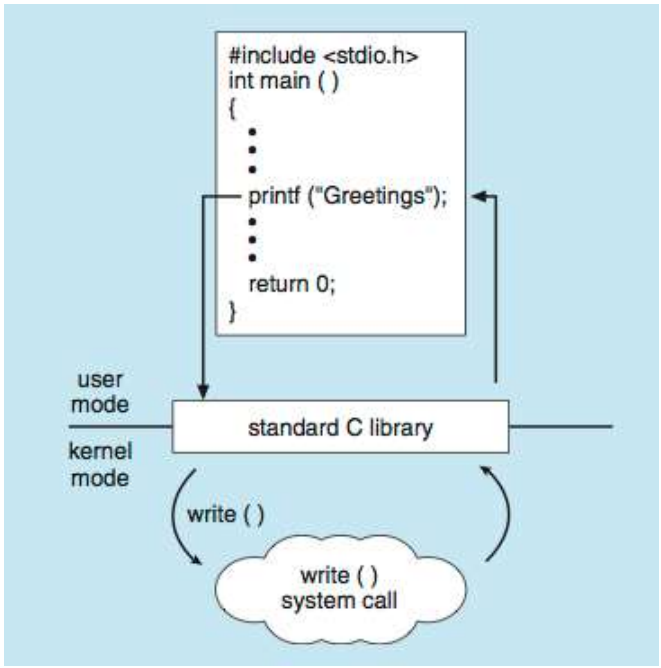
EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

	Windows	Unix
Process control	`CreateProcess()`	`fork()`
	`ExitProcess()`	`exit()`
	`WaitForSingleObject()`	`wait()`
File management	`CreateFile()`	`open()`
	`ReadFile()`	`read()`
	`WriteFile()`	`write()`
	`CloseHandle()`	`close()`
Device management	`SetConsoleMode()`	`ioctl()`
	`ReadConsole()`	`read()`
	`WriteConsole()`	`write()`
Information maintenance	`GetCurrentProcessID()`	`getpid()`
	`SetTimer()`	`alarm()`
	`Sleep()`	`sleep()`
Communications	`CreatePipe()`	`pipe()`
	`CreateFileMapping()`	`shm_open()`
	`MapViewOfFile()`	`mmap()`
Protection	`SetFileSecurity()`	`chmod()`
	`InitlializeSecurityDescriptor()`	`umask()`
	`SetSecurityDescriptorGroup()`	`chown()`



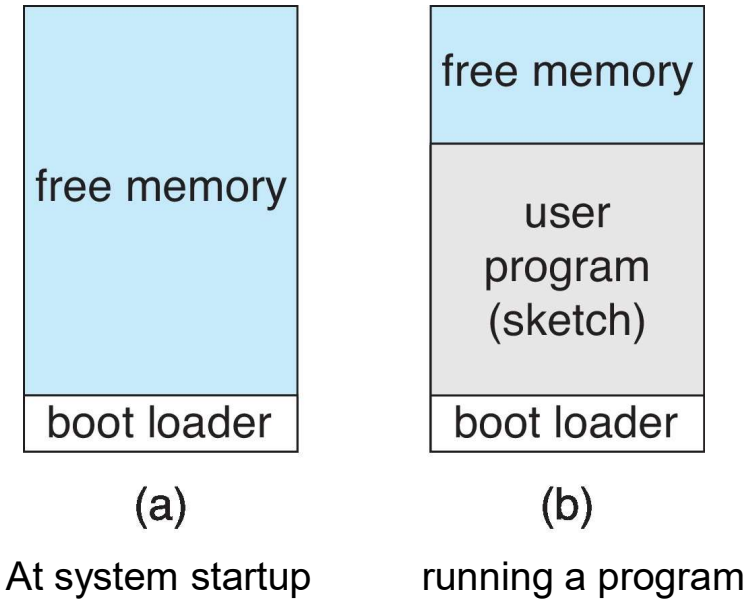
STANDARD C LIBRARY EXAMPLE

- C program invoking printf() library call, which calls write() system call



EXAMPLE: ARDUINO

- Single-tasking
- No operating system
- Programs (sketch) loaded via USB into flash memory
- Single memory space
- Boot loader loads program
- Program exit -> shell reloaded

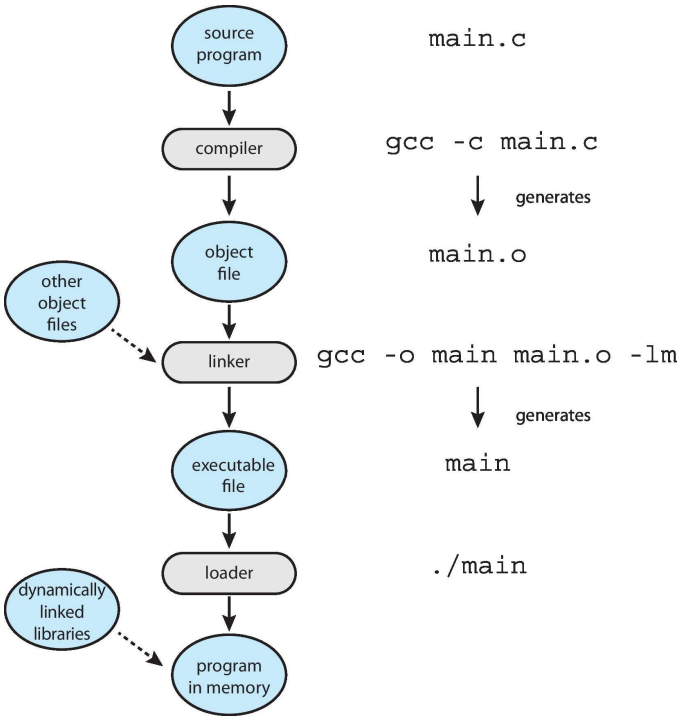


SYSTEM SERVICES

- **provide a convenient environment for program development and execution.**
  - **File manipulation**
  - **Status information sometimes stored in a file**
  - **Programming language support**
  - **Program loading and execution**
  - **Communications**
  - **Background services**
  - **Application programs**



LINKERS AND LOADERS



WHY APPLICATIONS ARE OPERATING SYSTEM SPECIFIC

- **Apps compiled on one system usually not executable on other OS**
- **Each OS provides its own unique system calls**
- **Some apps can be multi-operating system**



OS DESIGN AND IMPLEMENTATION

- **Internal structure of different OS can vary widely**
- **Start the design by defining goals and specifications**
- **Affected by choice of hardware, type of system**
- **User goals and System goals**
  - **User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast**
  - **System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient**



POLICY AND MECHANISM

- principle to separate

  Policy: ***What*** will be done?

  Mechanism: ***How*** to do it?

- Specifying and designing an OS is highly creative task of **software engineering**



IMPLEMENTATION

- Much variation
  - Early OSes in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- Actually usually a mix of languages
  - Lowest levels in assembly
  - Main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to **port** to other hardware
  - But slower
- **Emulation** can allow an OS to run on non-native hardware



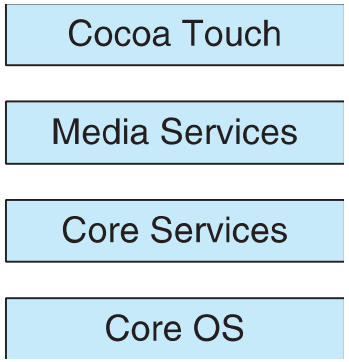
OPERATING SYSTEM STRUCTURE

- **General-purpose OS is very large program**
- **Various ways to structure ones**
  - **Simple structure – MS-DOS**
  - **More complex – UNIX**
  - **Layered - an abstraction**
  - **Microkernel**



IOS

- **Apple mobile OS for *iPhone*, *iPad***
  - **Structured on Mac OS X**
  - **Cocoa Touch Objective-C API for developing apps**
  - **Media services layer for graphics, audio, video**
  - **Core services provides cloud computing, databases**
  - **Core operating system, based on Mac OS X kernel**



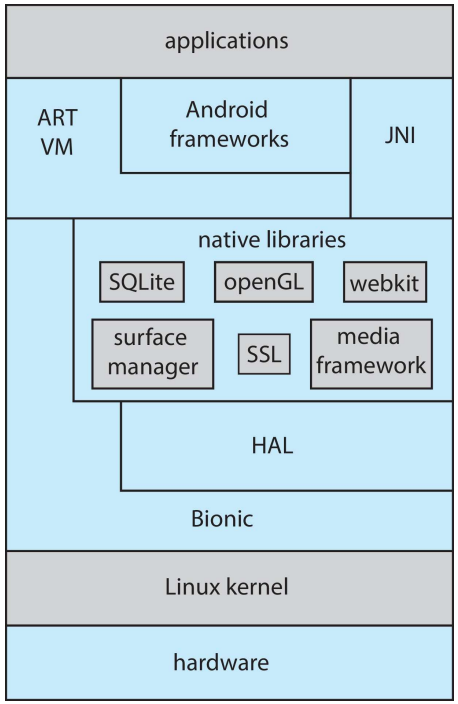


ANDROID

- Developed by Google (Open Source)
- Similar stack to iOS
- Based on Linux kernel
- Runtime environment includes core set of libraries and Dalvik virtual machine
- Libraries include frameworks
  - web browser (webkit)
  - database (SQLite)
  - multimedia



ANDROID ARCHITECTURE





OPERATING-SYSTEM DEBUGGING

- Finding and fixing errors
- OS generate **log files** containing error information
- Failure of an application can generate **core dump** file capturing memory of the process
- Operating system failure can generate **crash dump** file containing kernel memory



PERFORMANCE TUNING

- Improve performance by removing bottlenecks
- displaying measures of system behavior
- For example, "top" program or Windows Task Manager

